

# HADES

## Command-Line Flags — Cheat Sheet

Run with the 'py' launcher, e.g. py main.py [flags] · 2026-06-06

### All available flags

| Flag                   | Short     | Value   | What it does                                                                                                                                                                           |
|------------------------|-----------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>--url</b>           | <b>-u</b> | URL     | Target URL to scan. Must start with http:// or https://. If omitted, Hades asks for it interactively.                                                                                  |
| <b>--profile</b>       | <b>-p</b> | PROFILE | Which set of modules to run: quick, passive, cms, full, db_scan, ai_scan, engage, oob_scan. If omitted, an interactive menu is shown (menu option 8 = CVE Vulnerability Intelligence). |
| <b>--module</b>        | <b>-m</b> | NAME    | Run ONE module only (e.g. headers_check). Overrides --profile. Great for quick targeted checks.                                                                                        |
| <b>--output</b>        | <b>-o</b> | FORMAT  | Export the report to a file: json, html, or pdf. If omitted, results are only shown in the terminal.                                                                                   |
| <b>--proxy</b>         | <b>-</b>  | URL     | Route all traffic through an HTTP/HTTPS proxy, e.g. Burp Suite at http://127.0.0.1:8080.                                                                                               |
| <b>--threads</b>       | <b>-t</b> | N       | Number of concurrent worker threads (default: 10). Higher = faster but noisier/heavier.                                                                                                |
| <b>--ignore-robots</b> | <b>-</b>  | (flag)  | Ignore robots.txt Disallow rules during active scanning. Use only on targets you fully control.                                                                                        |
| <b>--exploit</b>       | <b>-</b>  | (flag)  | After the scan, launch sqlmap against any CONFIRMED SQL injection. Requires sqlmap. Authorised targets only.                                                                           |
| <b>--wordlist</b>      | <b>-w</b> | FILE    | Use a custom wordlist file instead of the built-in lists (affects dir_scan / admin_panel).                                                                                             |
| <b>--cookies</b>       | <b>-</b>  | STRING  | Send a Cookie header, e.g. "session=abc123; token=xyz". Use to scan as a logged-in user.                                                                                               |
| <b>--auth-token</b>    | <b>-</b>  | TOKEN   | Send an Authorization: Bearer <TOKEN> header. Use for API / token-protected targets.                                                                                                   |
| <b>--help</b>          | <b>-h</b> | (flag)  | Show the built-in help message listing all flags, then exit.                                                                                                                           |

### Scan profiles (--profile)

| Profile        | What it runs                                                                          |
|----------------|---------------------------------------------------------------------------------------|
| <b>quick</b>   | Fast surface scan: basic info, security headers, SSL/TLS, robots.txt.                 |
| <b>passive</b> | All reconnaissance + passive web checks. No active probing (no SQLi/XSS/brute-force). |
| <b>cms</b>     | CMS-focused: CMS detection, admin panels, and CVE mapping.                            |
| <b>full</b>    | Everything — the most thorough scan (this is the default behaviour).                  |

| Profile         | What it runs                                                                                                                                                                                                                                                              |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>db_scan</b>  | Red-team database security audit: DB ports, unauth access + data extraction, SQL/NoSQL injection, leaked secrets/connection strings, GraphQL, admin GUIs, dumps; emits a DB exposure score, attack path and loot. Pair with --exploit to launch sqlmap on confirmed SQLi. |
| <b>ai_scan</b>  | Red-team AI/LLM audit: SDK/provider fingerprint, exposed AI keys, unauthenticated local LLM servers, exposed AI UIs and the prompt-injection surface (OWASP LLM Top 10 + MITRE ATLAS).                                                                                    |
| <b>engage</b>   | Active exploitation engagement: confirms bugs then proves impact with benign payloads (RCE via id/uname, LFI read, SSRF to cloud metadata), writing evidence to loot/. Authorisation-gated.                                                                               |
| <b>oob_scan</b> | Out-of-band (OAST) blind-vuln detection: blind SSRF / RCE / stored XSS via a self-hosted callback listener; auto public tunnel (cloudflared/ngrok) so it works behind NAT.                                                                                                |
| <b>cve_scan</b> | CVE Vulnerability Intelligence (also interactive menu option 8): fingerprints the stack, matches real CVEs (2020+) from a local KEV/EPSS/NVD database, ranked by a Hades CVE Priority Score. Build the full offline corpus once with tools/build_vulndb.py.               |

## Module names (--module NAME)

Use any of these with **--module** to run just that one check:

**Recon:** `basic_info` · `whois_lookup` · `dns_check` · `ssl_check` · `port_scan` · `waf_detect` · `tech_stack`  
**Web:** `headers_check` · `robots_txt` · `sitemap` · `cms_detect` · `admin_panel` · `dir_scan` · `subdomain_scan` · `broken_links` · `http_methods` · `backup_files` · `sensitive_files` · `cookie_analysis` · `redirect_chain` · `email_exposure` · `favicon_hash` · `cors_check` · `clickjacking` · `dir_listing` · `blacklist_check` · `screenshot`  
**Vulns:** `sqli_detect` · `xss_detect` · `command_injection` · `ssti_detect` · `lfi_detect` · `open_redirect` · `ssrf_detect` · `cve_mapping` · `default_creds`  
**DB:** `db_security` (run via `--profile db_scan` – dedicated red-team database audit)  
**CVE:** `cve_vulnerability` (menu option 8 – CVE intelligence; build the corpus with `tools/build_vulndb.py`)

## Example commands

Interactive menu (asks URL + scan type)

```
py main.py
```

Quick scan of a site

```
py main.py --url https://example.com --profile quick
```

Full scan + HTML report

```
py main.py -u https://example.com -p full -o html
```

Run only one module

```
py main.py -u https://example.com -m headers_check
```

Scan through Burp Suite proxy

```
py main.py -u https://example.com --proxy http://127.0.0.1:8080
```

Authenticated scan (with a cookie)

```
py main.py -u https://example.com --cookies "session=abc123"
```

More threads (faster)

```
py main.py -u https://example.com -t 25
```

Detect + launch sqlmap on SQLi

```
py main.py -u "http://testaspnet.vulnweb.com/Comments.aspx?id=1" --exploit
```

Database security audit + HTML report

```
py main.py -u https://example.com -p db_scan -o html
```

DB audit + auto-exploit confirmed SQLi

```
py main.py -u http://testaspnet.vulnweb.com -p db_scan --exploit
```

CVE intelligence (interactive menu, option 8)

```
py main.py -u https://example.com
```

Build the full offline CVE corpus (once)

```
py tools/build_vulndb.py
```

Show all flags

```
py main.py --help
```